

Relatório Prático: Simulando Escalabilidade com Docker

Disciplina: Computação em Nuvem

Integrante: Wesley Azevedo Melo

1. Introdução

Este relatório apresenta os resultados práticos e a análise teórica do Exercício 2 da disciplina de Computação em Nuvem. O objetivo principal do laboratório foi simular e analisar o comportamento de um ambiente escalável utilizando containers Docker administrados via Docker Compose.

A atividade consistiu no deploy de um servidor web Nginx configurado para distribuir requisições em múltiplas réplicas, permitindo observar diretamente conceitos fundamentais da arquitetura de nuvem, tais como o escalonamento horizontal (Scale Out e Scale In), alta disponibilidade, tolerância a falhas e comportamento do hardware (uso de CPU) sob condições de estresse.

2. Configuração da Infraestrutura

Para a execução do ambiente, foi estruturado o arquivo de especificação abaixo, configurado para expor a porta padrão do servidor web a conexões dinâmicas do host, evitando conflitos de portas entre as réplicas simultâneas:

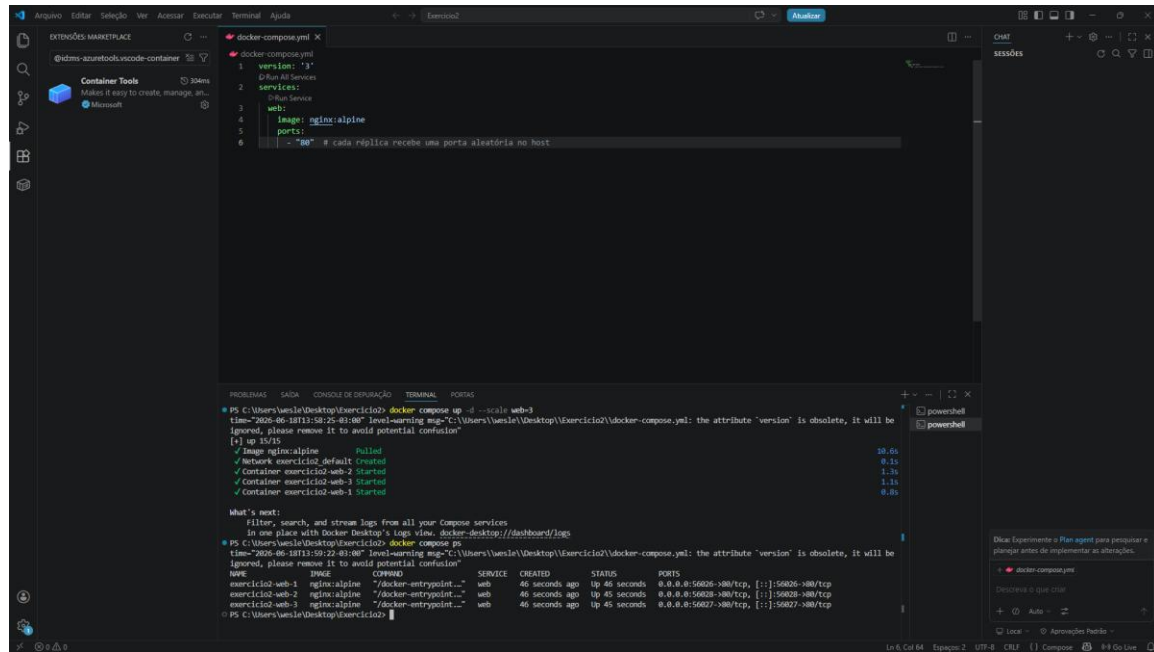
```
version: '3'
services:
  web:
    image: nginx:alpine
    ports:
      - "80" # Cada réplica recebe uma porta aleatória no host
```

3. Evidências Práticas (Capturas de Tela)

Capturas de tela obtidas durante a execução dos comandos e testes de estresse no terminal e no Docker Desktop:

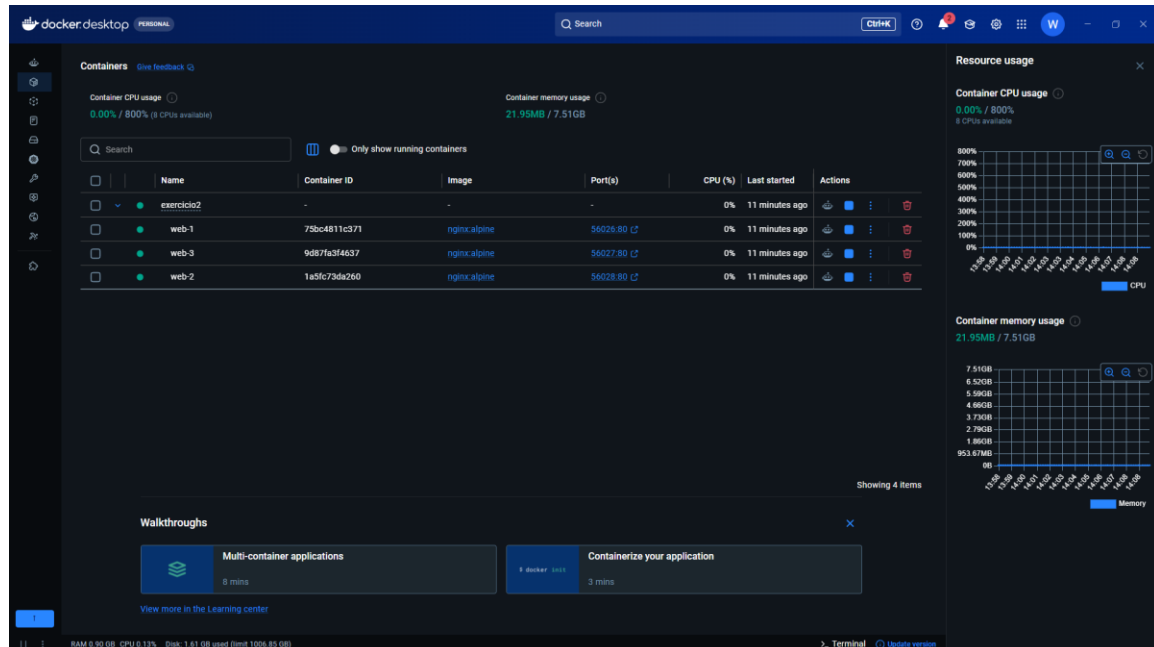
3.1 Validação do Escalonamento de Réplicas

[Print 1]



Evidência do comando `docker compose ps` com 3 réplicas ativas e suas respectivas portas aleatórias.

[Print 2]



Evidência do comando `docker compose ps` com 5 réplicas ativas após o aumento de demanda.

[Print 3]

The image shows a development environment with VS Code and Docker Desktop. In VS Code, a `docker-compose.yml` file is open, showing a service named `web` using the `nginx:alpine` image. The terminal shows the command `docker compose up --scale web=5` being executed, resulting in 5 containers being started. The Docker Desktop interface shows the 'Containers' tab with a table of running containers. The 'Resource usage' panel on the right shows CPU and memory usage for the containers.

docker-compose.yml

```
version: '3'
services:
  web:
    image: nginx:alpine
    ports:
      - "80"
    # cada réplica recebe uma porta aleatória no host
```

Terminal Output:

```
PS C:\Users\wansle\Desktop\exercicio2> docker compose up --scale web=5
time="2025-06-18T14:18:00-03:00" level=warning msg="C:\Users\wansle\Desktop\exercicio2\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] up 5/5
✓ Container exercicio2-web-1 Running
✓ Container exercicio2-web-2 Running
✓ Container exercicio2-web-3 Running
✓ Container exercicio2-web-5 Started
✓ Container exercicio2-web-4 Started

What's next:
Filter, search, and stream logs from all your Compose services
In one place with Docker Desktop's logs view. docker-desktop://dashboard/logs
PS C:\Users\wansle\Desktop\exercicio2> docker compose ps
time="2025-06-18T14:18:15-03:00" level=warning msg="C:\Users\wansle\Desktop\exercicio2\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
name                image               command             status    CREATED             STATUS             PORTS
exercicio2-web-1    nginx:alpine        /docker-entrypoint... web        11 minutes ago    up 11 minutes    0.0.0.0:56026->80/tcp, [...]
exercicio2-web-2    nginx:alpine        /docker-entrypoint... web        11 minutes ago    up 11 minutes    0.0.0.0:56028->80/tcp, [...]
exercicio2-web-3    nginx:alpine        /docker-entrypoint... web        11 minutes ago    up 11 minutes    0.0.0.0:56027->80/tcp, [...]
exercicio2-web-4    nginx:alpine        /docker-entrypoint... web        15 seconds ago    up 14 seconds    0.0.0.0:65068->80/tcp, [...]
exercicio2-web-5    nginx:alpine        /docker-entrypoint... web        15 seconds ago    up 13 seconds    0.0.0.0:65069->80/tcp, [...]
```

Docker Desktop - Containers

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
exercicio2	-	-	-	0%	35 seconds ago	
web-1	78bc4811c371	nginx:alpine	56026.80 C	0%	12 minutes ago	
web-3	9d87fa3f4637	nginx:alpine	56027.80 C	0%	12 minutes ago	
web-2	1a5fc73d6260	nginx:alpine	56028.80 C	0%	12 minutes ago	
web-4	3d36117a4242	nginx:alpine	65068.80 C	0%	36 seconds ago	
web-5	b56046631004	nginx:alpine	65069.80 C	0%	35 seconds ago	

Resource usage

Container CPU usage: 0.00% / 800% (8 CPUs available)

Container memory usage: 37.78MB / 7.51GB

Showing 6 items

Walkthroughs: Multi-container applications (8 mins), Containerize your application (3 mins)

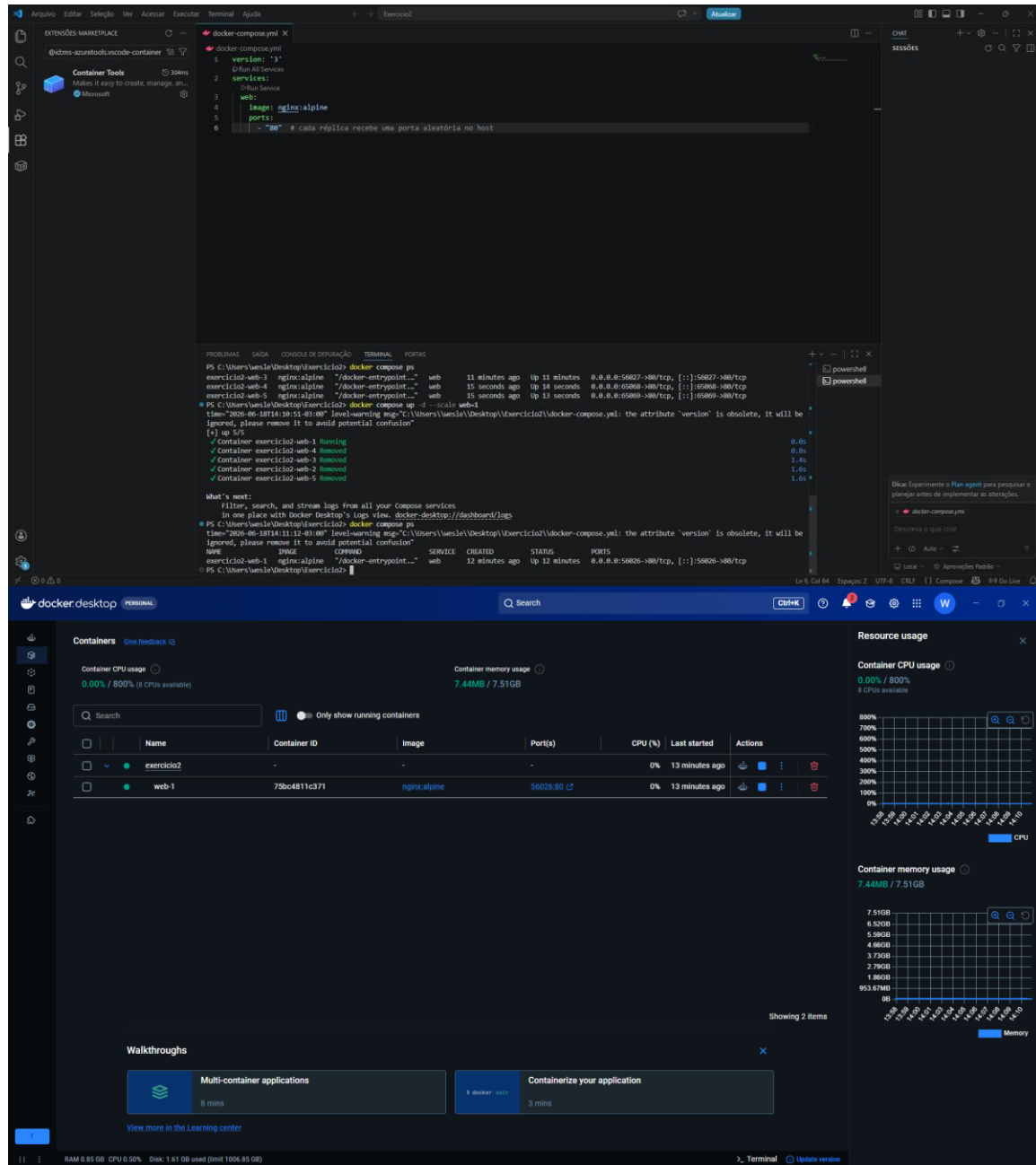
View more in the Learning center

RAM 5.95 GB CPU 1.00% Disk 1.61 GB used (limit 1008.85 GB)

Evidência do comando `docker compose ps` com apenas 1 réplica ativa após a redução de recursos.

3.2 Monitoramento de Carga e Recursos

[Print 4]



Evidência do Docker Desktop (aba Stats) ou do comando `docker stats` exibindo o pico no uso de CPU% durante a execução do laço de 500 requisições via PowerShell.

4. Análise e Respostas Teóricas

1. Qual a diferença entre escalabilidade vertical e horizontal?

A escalabilidade vertical (Scale Up) consiste em adicionar mais recursos de hardware a um único servidor já existente, como expandir a memória RAM, trocar o processador por um com mais núcleos ou aumentar a capacidade do disco rígido. É uma abordagem limitada pelo teto físico da máquina e costuma gerar downtime durante o upgrade.

A escalabilidade horizontal (Scale Out) baseia-se na adição de novas máquinas ou instâncias (como containers) para trabalharem em paralelo dentro do ecossistema. Em vez de tornar um único nó mais robusto, distribui-se a carga de trabalho entre múltiplos nós menores trabalhando de forma cooperativa, permitindo um crescimento virtualmente ilimitado.

2. O que você acabou de fazer foi escalabilidade vertical ou horizontal? Por quê?

O procedimento realizado neste laboratório configurou uma escalabilidade horizontal (processos de Scale Out ao subir para 5 réplicas e Scale In ao reduzir para 1).

Por quê? Em nenhum momento as configurações de hardware, limites de CPU ou memória de um container individual foram alterados. O crescimento e suporte à demanda foram alcançados estritamente através do aumento do número de instâncias idênticas (containers adicionais) rodando concorrentemente e dividindo o volume de requisições web geradas pelo script.

3. O que aconteceria se um dos containers falhasse com 3 réplicas? E com apenas 1?

- Com 3 Réplicas: O ambiente apresenta Alta Disponibilidade e Tolerância a Falhas. Caso um dos containers pare de responder ou sofra uma falha crítica, as outras 2 réplicas saudáveis continuam ativas e assumem o fluxo total de dados. Para o usuário externo, o impacto é nulo ou imperceptível.

- Com apenas 1 Réplica: O cenário introduz um Ponto Único de Falha (Single Point of Failure). Sem instâncias de redundância, o colapso do único container ativo derruba o ecossistema imediatamente, deixando a aplicação completamente fora do ar (downtime) até que um novo container seja manualmente instanciado.